
A Zeroth-Order Block Coordinate Descent Algorithm for Huge-Scale Black-Box Optimization

HanQin Cai¹ Yuchen Lou² Daniel McKenzie¹ Wotao Yin³

Abstract

We consider the zeroth-order optimization problem in the huge-scale setting, where the dimension of the problem is so large that performing even basic vector operations on the decision variables is infeasible. In this paper, we propose a novel algorithm, coined ZO-BCD, that exhibits favorable overall query complexity *and* has a much smaller per-iteration computational complexity. In addition, we discuss how the memory footprint of ZO-BCD can be reduced even further by the clever use of circulant measurement matrices. As an application of our new method, we propose the idea of crafting adversarial attacks on neural network based classifiers in a *wavelet domain*, which can result in problem dimensions of over one million. In particular, we show that crafting adversarial examples to audio classifiers in a wavelet domain can achieve the state-of-the-art attack success rate of 97.9% with significantly less distortion.

1. Introduction

We are interested in problem (1) under the restrictive assumption that one only has noisy zeroth-order access to f (*i.e.* one cannot access the gradient, ∇f) *and* the dimension of the problem, d , is huge, say $d > 10^7$.

$$\underset{x \in \mathcal{X} \subset \mathbb{R}^d}{\text{minimize}} f(x). \quad (1)$$

Such problems (with small or large d) arise frequently in domains as diverse as simulation-based optimization in chemistry and physics (Reeja-Jayan et al., 2012), hyperparameter tuning for combinatorial optimization solvers (Hutter et al., 2014) and for neural networks (Bergstra & Bengio, 2012)

¹Department of Mathematics, University of California, Los Angeles, Los Angeles, CA, USA ²University of Hong Kong, Hong Kong SAR, China ³DAMO Academy, Alibaba US, Bellevue, WA, USA. Correspondence to: HanQin Cai <hqcai@math.ucla.edu>.

and online marketing (Flaxman et al., 2005). Lately, algorithms for zeroth-order optimization have drawn increasing attention due to their use in finding good policies in reinforcement learning (Salimans et al., 2017; Mania et al., 2018; Choromanski et al., 2020) and in crafting adversarial examples given only black-box access to neural-network based classifiers (Chen et al., 2017; Lian et al., 2016; Alzantot et al., 2018; Cai et al., 2020b). We note that in all of these applications queries (*i.e.* evaluating f at a chosen point) are considered expensive, thus it is desirable for zeroth-order optimization algorithms to be as *query efficient* as possible.

Unfortunately, it is known (Jamieson et al., 2012) that the worst case query complexity of *any* noisy zeroth order algorithm for strongly convex f scales linearly with d . Clearly, this is prohibitive for huge d . Recent works have begun to side-step this issue by assuming f has additional, low-dimensional, structure. For example, (Wang et al., 2018; Balasubramanian & Ghadimi, 2018; Cai et al., 2020a;b) assume the gradients ∇f are (approximately) s -sparse (see Assumption 5) while (Golovin et al., 2019) and others assume $f(x) = g(Az)$ where $A : \mathbb{R}^s \rightarrow \mathbb{R}^d$ and $s \ll d$. All of these works promise a query complexity that scales linearly with the intrinsic dimension, s , and only logarithmically with the extrinsic dimension, d . However there is no free lunch here; the improved complexity of (Golovin et al., 2019) requires access to *noiseless* function evaluations, the results of (Balasubramanian & Ghadimi, 2018) only hold if the support of $\nabla f(x)$ is *fixed*¹ for all $x \in \mathcal{X}$ and while (Wang et al., 2018; Cai et al., 2020a;b) allow for noisy function evaluations and changing gradient support, both solve a computationally intensive optimization problem as a sub-routine, requiring at least $\Omega(sd \log(d))$ memory and FLOPS per iteration.

1.1. Contributions

In this paper we provide the first zeroth-order optimization algorithm enjoying a sub-linear (in d) query complexity *and* a sub-linear per-iteration computational complexity. In addition, our algorithm has an exceptionally small memory footprint. Furthermore, it does not require the repeated sampling of d -dimensional random vectors, a hallmark of

¹See Appendix A of (Cai et al., 2020b) for a proof of this.

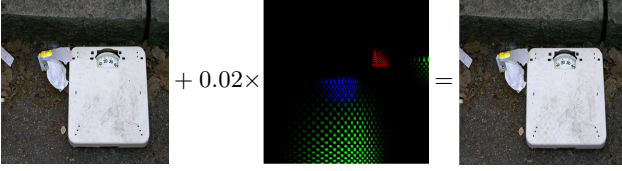


Figure 1. **Left:** Original image from ImageNet with true label “scale”. **Center:** Wavelet perturbation crafted using ZO-BCD. **Right:** The attacked image, constructed by adding the perturbation, scaled down by 0.02, to the original image. Mis-classified as a “switch”.

many zeroth-order optimization algorithms. With this new algorithm, ZO-BCD, in hand we are able to solve black-box optimization problems of a size hitherto unimagined. Specifically, we consider the problem of generating adversarial examples to fool neural-network-based classifiers, given only black-box access to the model (as introduced in (Chen et al., 2017)). However, we consider generating these malicious examples by perturbing natural examples *in a wavelet domain*. For image classifiers (we consider Inception-v3 trained on ImageNet) we are able to produce attacked images with a record low ℓ_2 distortion of 13.7 and a success rate of 96%, exceeding the state of the art. For audio classifiers, switching to a wavelet domain results in a problem dimension of over 1.7 million. Using ZO-BCD, this is not an issue and we achieve a targeted attack success rate of 97.93% with a mean distortion of -6.32 dB.

1.2. Relation to prior work

As mentioned above, the recent works (Wang et al., 2018; Balasubramanian & Ghadimi, 2018; Cai et al., 2020b) provide zeroth-order algorithms whose query complexity scales linearly with s and logarithmically with d . In order to ameliorate the prohibitive computational and memory cost associated with huge d , several domain-specific heuristics have been employed in the literature. For example in (Chen et al., 2017; Alzantot et al., 2019), in relation to adversarial attacks, an upsampling operator $D : \mathbb{R}^p \rightarrow \mathbb{R}^d$ with $p \ll d$ is employed. Problem (1) is then replaced with the lower dimensional problem: $\text{minimize}_{z \in \mathbb{R}^p} f(D(z))$. Several other works (Alzantot et al., 2018; Taori et al., 2019; Cai et al., 2020b) choose a low dimensional random subspace $T_k \subset \mathbb{R}^d$ at each iteration and then restrict $x_{k+1} - x_k \in T_k$. We emphasize that none of the aforementioned works *prove* such a procedure will converge, and our work is partly motivated by the desire to provide this empirically successful trick with firm guarantees of success.

In the reinforcement learning literature it is common to evaluate the $f(x_k + z_{k,i})$ on parallel devices and send the computed function value and the perturbation $z_{k,i}$ to a central worker, which then computes x_{k+1} . As $x \in \mathbb{R}^d$

parametrizes a neural network, d can be extremely large, and hence the communication of the $z_{k,i}$ between workers becomes a bottle neck. (Salimans et al., 2017) overcomes this with a “seed sharing” trick, but again this heuristic lacks rigorous analysis. We hope ZO-BCD’s (particularly the ZO-BCD-RC variant, see Section 3) intrinsically small memory footprint will make it a competitive, principled alternative.

Finally, although two recent works have examined the idea of wavelet domain adversarial attacks (Anshuman et al., 2020; Din et al., 2020) they are of a very different nature to our approach. We discuss this further in Section 4.

1.3. Assumptions and notation

As mentioned, we will suppose the decision variables x have been subdivided into J blocks of sizes $d^{(1)}, \dots, d^{(J)}$. Following the notation of (Tappenden et al., 2016), we suppose there exists a permutation matrix $U \in \mathbb{R}^{d \times d}$ and a division of U into submatrices $U = [U^{(1)}, U^{(2)}, \dots, U^{(J)}]$ such that $U^{(j)} \in \mathbb{R}^{d \times d^{(j)}}$ and the j -th block is spanned by the columns of $U^{(j)}$. Letting $x^{(j)}$ denote the decision variables in the j -th block, we write $x = \sum_{j=1}^J U^{(j)} x^{(j)}$ or simply $x = (x^{(1)}, x^{(2)}, \dots, x^{(J)})$. We shall consistently use the notation $g(x) := \nabla f(x)$, omitting x if it is clear from context. By $g^{(j)}$ we mean the components of the gradient corresponding to the j -th block, i.e. $g^{(j)} = \nabla_{x^{(j)}} f$, regarded as either a vector in $\mathbb{R}^d$ or in $\mathbb{R}^{d^{(j)}}$. Finally, we use $\tilde{O}(\cdot)$ notation to suppress logarithmic factors. Let us now introduce some standard assumptions on the objective function.

Assumption 1 (Block Lipschitz differentiability). f is continuously differentiable and for some fixed constant L_j

$$\|g^{(j)}(x) - g^{(j)}(x + U^{(j)}t)\|_2 \leq L_j \|t\|_2$$

for all $j = 1, \dots, J$, $x \in \mathcal{X}$ and $t \in \mathbb{R}^{d^{(j)}}$.

If f is L -Lipschitz differentiable then it is also block Lipschitz differentiable, with $\max_j L_j \leq L$.

Assumption 2 (Convexity). $\mathcal{X}$ is a convex set, and $f(tx + (1-t)y) \leq tf(x) + (1-t)f(y)$ for all $x, y \in \mathcal{X}$, $t \in [0, 1]$.

Define the solution set $\mathcal{X}^* = \arg \min_{x \in \mathcal{X}} f(x)$. If this set is non-empty we define the level set radius for $x \in \mathcal{X}$ as:

$$\mathcal{R}(x) := \max_{y \in \mathcal{X}} \max_{x^* \in \mathcal{X}^*} \{\|y - x^*\|_2 : f(y) \leq f(x)\}. \quad (2)$$

Assumption 3 (Non-empty solution set and Bounded level sets). $\mathcal{X}^*$ is non-empty and $\mathcal{R}(x_0) < \infty$.

Assumption 4 (Adversarially noisy oracle). f is only accessible through a noisy oracle: $E_f(x) = f(x) + \xi$, where ξ is a random variable satisfying $|\xi| \leq \sigma$.

Assumption 5 (Sparse gradients). *There exists a fixed integer $0 < s_{\text{exact}} < d$ such that for all $x \in \mathbb{R}^d$:*

$$\|g(x)\|_0 := |\{i : g_i(x) \neq 0\}| \leq s_{\text{exact}}.$$

It is of interest to relax this assumption to an ‘‘approximately sparse’’ assumption, such as in (Cai et al., 2020b). However, it is unclear randomly chosen blocks (see Section 2.1) will inherit this property. We leave the analysis of this case for future work. Finally, let $\nabla_{jj}^2 f \in \mathbb{R}^{d^{(j)} \times d^{(j)}}$ denote the j -th block Hessian.

Assumption 6 (Weakly sparse block Hessian). *f is twice differentiable and, for all $j = 1, \dots, J$, $x \in \mathcal{X}$ we have $\|\nabla_{jj}^2 f(x)\|_1 \leq H$ for some fixed constant H .*

Note that $\|\cdot\|_1$ represents the *element-wise* ℓ_1 -norm: $\|B\|_1 = \sum_{i,j} |B_{ij}|$.

2. Gradient estimators

Randomized (block) coordinate descent methods are an attractive alternative to full gradient methods for huge-scale problems (Nesterov, 2012). ZO-BCD is a block coordinate method adapted to the zeroth-order setting and conceptually has three steps:

1. Choose a block, $j \in \{1, \dots, J\}$ at random.
2. Use zeroth-order queries to find an approximation $\hat{g}_k^{(j)}$ of the true block gradient $g_k^{(j)}$.
3. Take a negative gradient step: $x_{k+1} = x_k - \alpha \hat{g}_k^{(j)}$.

We abuse notation slightly; the block gradient $\hat{g}_k^{(j)}$ is regarded as both a vector in $\mathbb{R}^{d^{(j)}}$ and a vector in $\mathbb{R}^d$ with non-zeros in the j -th block only.

In principle any scheme for constructing an estimator of g_k could be adapted for estimating $g_k^{(j)}$, as long as one is able to bound $\|g_k^{(j)} - \hat{g}_k^{(j)}\|_2$ with high probability. As we wish to exploit gradient sparsity, we choose to adapt the estimator presented in (Cai et al., 2020b). Let us now discuss how to do so. Fix a sampling radius $\delta > 0$. Suppose the j -th block has been selected and choose m sample directions $z_1, \dots, z_m \in \mathbb{R}^{d^{(j)}}$ from a Rademacher distribution². Consider the finite difference approximations to the directional derivatives:

$$y_i = \frac{1}{\sqrt{m}} \frac{E_f(x + \delta U^{(j)} z_i) - E_f(x)}{\delta} \approx \frac{1}{\sqrt{m}} z_i^\top g^{(j)} \quad (3)$$

Stack the y_i into a vector $y \in \mathbb{R}^m$, let $Z \in \mathbb{R}^{m \times (d/J)}$ be the matrix with rows $z_i^\top / \sqrt{m}$ and observe $y \approx Z g^{(j)}$; an

²That is, the entries of z_i are $+1$ or -1 with equal probability.

Algorithm 1 Block Gradient Estimation

- 1: **Input:** x : current point; j : choice of block; s : gradient sparsity level; δ : query radius; n : number of CoSaMP iterations; $\{z_i\}_{i=1}^m$: sample directions in $\mathbb{R}^{d^{(j)}}$.
 - 2: **for** $i = 1$ **to** m **do**
 - 3: $y_i \leftarrow (E_f(x + \delta U^{(j)} z_i) - E_f(x)) / (\sqrt{m} \delta)$
 - 4: **end for**
 - 5: $\mathbf{y} \leftarrow [y_1, \dots, y_m]^T$; $Z \leftarrow 1/\sqrt{m} [z_1, \dots, z_m]^T$
 - 6: $\hat{g}^{(j)} \approx \arg \min_{\|v\|_0 \leq s} \|Zv - \mathbf{y}\|_2$ by n iterations of CoSaMP
 - 7: **Output:** $\hat{g}^{(j)}$: estimated block gradient.
-

underdetermined linear system. If³ $\|g\|_0 \leq s$ (see Assumption 5) then also $\|g^{(j)}\|_0 \leq s$. Thus, we approximate $g^{(j)}$ by solving the sparse recovery problem:

$$\hat{g}^{(j)} = \arg \min \|Zv - y\|_2 \quad v \in \mathbb{R}^{d^{(j)}} \text{ and } \|v\|_0 \leq s. \quad (4)$$

We propose solving Problem (4) using n iterations of CoSaMP (Needell & Tropp, 2009), but other choices are possible. This approach, presented as Algorithm 1, yields an accurate gradient estimator (Cai et al., 2020b) using only $\tilde{\mathcal{O}}(s)$ queries, assuming $g^{(j)}$ is sparse. In contrast, direct finite differencing (Berahas et al., 2019) requires $\mathcal{O}(d^{(j)})$ queries.

Theorem 2.1. *Suppose f satisfies Assumptions 1, 5 and 6. Let $g^{(j)}$ be the output of Algorithm 1 with $\delta = 2\sqrt{\sigma/H}$, $s \geq s_{\text{exact}}$ and $m = b_1 s \log(d/J)$ Rademacher sample directions. Then with probability at least $1 - (s/d)^{b_2 s}$:*

$$\|\hat{g}^{(j)} - g^{(j)}\|_2 \leq \rho^n \|g^{(j)}\|_2 + 2\tau\sqrt{\sigma H}. \quad (5)$$

The constants b_1 and b_2 are directly proportional; more sample directions results in a higher probability of recovery. In our experiments we consider $1 \leq b_1 \leq 4$. The constant ρ and τ arise from the analysis of CoSaMP. Both are inversely proportional to b_1 . For our range of b_1 , $\rho \approx 0.5$ and $\tau \approx 10$.

2.1. Almost equisparsed blocks using randomization

Suppose f satisfies Assumption 5, so $\|g(x)\|_0 \leq s_{\text{exact}}$ for all x . In general one cannot improve upon the bound $\|g^{(j)}(x)\|_0 \leq s_{\text{exact}}$; perhaps all non-zero entries of g lie in the j -th block. However, by *randomizing* the blocks one can guarantee, with high probability, the non-zero entries of g are *almost equally distributed* over the J blocks. We assume, for simplicity, equal-sized blocks (*i.e.* $d^{(j)} = d/J$).

Theorem 2.2. *Choose U uniformly at random. For any $\Delta > 0$, $x \in \mathbb{R}^d$ we have $\|g^{(j)}(x)\|_0 \leq (1 + \Delta)s_{\text{exact}}/J$ for all j with probability at least $1 - 2J \exp(-\frac{\Delta^2 s_{\text{exact}}}{3J})$.*

³Throughout, we assume $s \geq s_{\text{exact}}$ is specified by the user.

It will be convenient to fix a value of Δ , say $\Delta = 0.1$. An immediate consequence of Theorem 2.2 is that one can improve upon the query complexity of Theorem 2.1:

Corollary 2.3. *Choose U uniformly at random. For fixed $x \in \mathbb{R}^d$ the error bound (5) in Theorem 2.1 still holds, now with probability $1 - \mathcal{O}\left(J \exp\left(\frac{-0.01s_{\text{exact}}}{3J}\right)\right)$, for $s := s_{\text{block}} \geq 1.1s_{\text{exact}}/J$ (and all other parameters the same).*

This allows us to use approximately J times fewer queries per iteration.

2.2. Further reducing the required randomness

As discussed in (Cai et al., 2020b), one favorable feature of using a compressed sensing based gradient estimator is the error bound (5) is *universal*. That is, it holds for all $x \in \mathbb{R}^d$ for the *same set of sample directions* $\{z_i\}_{i=1}^m \subset \mathbb{R}^{d/J}$. So, instead of resampling new vectors at each iteration we may use *the same* sampling directions *for each block and each iteration*. Thus, only $md/J = \tilde{\mathcal{O}}(s_{\text{exact}}d/J^2)$ binary random variables need to be sampled, stored and transmitted in ZO-BCD. Remarkably, one can do even better by choosing as sample directions a subset of the rows of a circulant matrix. Recall a circulant matrix of size $d/J \times d/J$, generated by $v \in \mathbb{R}^{d/J}$, has the following form:

$$\mathcal{C}(v) = \begin{pmatrix} v_1 & v_2 & \cdots & v_{d/J} \\ v_{d/J} & v_1 & \cdots & v_{d/J-1} \\ \vdots & \vdots & \ddots & \vdots \\ v_2 & \cdots & v_{d/J} & v_1 \end{pmatrix}. \quad (6)$$

Equivalently, $\mathcal{C}(v)$ is the matrix with rows $\mathcal{C}_i(v)$ where:

$$\mathcal{C}_i(v) \in \mathbb{R}^{d/J} \quad \text{and} \quad \mathcal{C}_i(v)_j = v_{i+j-1}.$$

By exploiting recent results in signal processing, we show:

Theorem 2.4. *Assign blocks randomly as in Corollary 2.3. Let $z \in \mathbb{R}^{d/J}$ be a Rademacher random vector. Fix $s := s_{\text{block}} \geq 1.1s_{\text{exact}}/J$. Choose a random subset $\Omega = \{j_1, \dots, j_m\} \subset \{1, \dots, d/J\}$ of cardinality $m = b_3 s \log^2(s) \log^2(d/J)$ and let $z_i = \mathcal{C}_{j_i}(z)$ for $i = 1, \dots, m$. Let $\delta = 2\sqrt{\sigma/H}$. For fixed $x \in \mathbb{R}^d$ the error bound (5) in Theorem 2.1 still holds, now with probability at least*

$$1 - 2J \exp\left(\frac{-0.01s_{\text{exact}}}{3J}\right) - (d/J)^{\log(d/J) \log^2(4s)}.$$

Hence, one only needs d/J binary random variables (to construct z) and $m = \tilde{\mathcal{O}}(s_{\text{exact}}/J)$ randomly selected integers for the *entire* algorithm. Note (partial) circulant matrices allow for a *fast multiplication*, further reducing the computational complexity of Algorithm 1.

Algorithm 2 ZO-BCD

```

1: Input:  $x_0$ : initial point;  $s$ : gradient sparsity level;  $\alpha$ :
   step size;  $\delta$ : query radius;  $J$ : number of blocks.
2:  $s_{\text{block}} \leftarrow 1.1s/J$ 
3: Randomly divide  $x$  into  $J$  equally sized blocks.
4: if ZO-BCD-R then
5:    $m \leftarrow b_1 s_{\text{block}} \log(d/J)$ 
6:   Generate Rademacher random vectors  $z_1, \dots, z_m$ 
7: else if ZO-BCD-RC then
8:    $m \leftarrow b_3 s_{\text{block}} \log^2(s_{\text{block}}) \log^2(d/J)$ 
9:   Generate Rademacher random vector  $z$ .
10:  Randomly choose  $\Omega \subset \{1, \dots, d/J\}$  with  $|\Omega| = m$ 
11:  Let  $z_i = \mathcal{C}_{j_i}(z)$  for  $i = 1, \dots, m$  and  $j_i \in \Omega$ 
12: end if
13: for  $k = 0$  to  $K$  do
14:   Select a block  $j \in \{1, \dots, J\}$  uniformly at random.
15:    $\hat{g}_k^{(j)} \leftarrow \text{Gradient Estimation}(x_k^{(j)}, s_{\text{block}}, \delta, \{z_i\}_{i=1}^m)$ 
16:    $x_{k+1} \leftarrow x_k - \alpha \hat{g}_k^{(j)}$ 
17: end for
18: Output:  $x_K$ : estimated optimum point.
    
```

3. The proposed algorithm: ZO-BCD

Let us now introduce our new algorithm. We consider two variants, distinguished by the kind of sampling directions used. ZO-BCD-R uses **Rademacher** sampling directions. ZO-BCD-RC uses **Rademacher-Circulant** sampling directions, as described in Section 2.2. For simplicity, we present our algorithm for randomly selected, equally sized coordinate blocks. With minor modifications our results still hold for user-defined and/or unequally sized blocks (see Appendix B). The following theorem guarantees both variants converge at a sublinear rate to within a certain error tolerance. As the choice of block in each iteration is random, our results are necessarily probabilistic. We say x_K is an ε -optimal solution if $f(x_K) - f^* \leq \varepsilon$.

Theorem 3.1. *Assume f satisfies Assumptions 1–6. Define:*

$$L_{\max} = \max_j L_j \quad \text{and} \quad c_1 = 2JL_{\max}\mathcal{R}^2(x_0).$$

Assume $4\rho^{4n} + \frac{16\tau^2\sigma H}{c_1 L_{\max}} < 1$. Choose sparsity $s \geq s_{\text{exact}}$, step size $\alpha = \frac{1}{L_{\max}}$ and query radius $\delta = 2\sqrt{\sigma/H}$. Choose the number of CoSaMP iterations n and error tolerance ε such that:

$$\frac{c_1}{2} \left(2\rho^{2n} + \sqrt{4\rho^{4n} + \frac{16\tau^2\sigma H}{c_1 \zeta L_{\max}}} \right) < \varepsilon < f(x_0) - f^*.$$

With probability at least $1 - \zeta - \tilde{\mathcal{O}}\left(\frac{J^2}{\varepsilon} \exp\left(\frac{-0.01s_{\text{exact}}}{3J}\right)\right)$, ZO-BCD-R finds an ε -optimal solution in $\tilde{\mathcal{O}}(s/\varepsilon)$ queries, requiring $\tilde{\mathcal{O}}(sd/J^2)$ FLOPS per iteration and $\tilde{\mathcal{O}}(sd/J^2)$

total memory. With probability at least

$$1 - \tilde{\mathcal{O}}\left(\frac{J^2}{\varepsilon} \exp\left(-\frac{0.01s_{\text{exact}}}{3J}\right)\right) - (d/J)^{\log(d/J) \log^2(4.4s/J)}$$

ZO-BCD-RC finds an ε -optimal solution using $\tilde{\mathcal{O}}(s/\varepsilon)$ queries, $\tilde{\mathcal{O}}(d/J)$ FLOPS per iteration and $\mathcal{O}(d/J)$ total memory.

Thus, up to logarithmic factors, ZO-BCD achieves the same query complexity as ZORO (Cai et al., 2020b) with a much lower per-iteration computational cost. We pay for the improved computational and memory complexity of ZO-BCD-RC with a slightly worse theoretical query complexity (by a logarithmic factor) due to the requirements of Theorem 2.4. First order block coordinate descent methods typically have a probability of success $1 - \zeta$, thus in switching to zeroth-order this probability decreases by a factor which is negligible for truly huge problems (e.g. $d \approx 10^6$ and $s_{\text{exact}} \approx 10^4$). For smaller problems we find randomly re-assigning the decision variables to blocks every J iterations is a good way to increase ZO-BCD’s probability of success.

4. Sparse wavelet transform attacks

Adversarial attacks on neural network based classifiers is a popular application and benchmark for zeroth-order optimizers (Chen et al., 2017; 2019; Ilyas et al., 2018; Modas et al., 2019). Specifically, let $F(x) \in [0, 1]^C$ denote the predicted probabilities returned by the model for input signal x . Then the goal is to find a small distortion δ such that that the model’s top-1 prediction on $x + \delta$ is no longer correct: $\arg\max_{c=1,\dots,C} F_c(x + \delta) \neq \arg\max_{c=1,\dots,C} F_c(x)$. Because we only have access to the logits, $F(x)$, not the internal workings of the model, we are unable to compute $\nabla F(x)$ and hence this problem is of zeroth order. Recently, (Cai et al., 2020b) showed it is reasonable to assume the attack loss function exhibits (approximate) gradient sparsity, and proposed generating adversarial examples by adding a distortion to the victim image that is sparse in the image pixel domain. We extend this and propose a novel *sparse wavelet transform attack*, which searches for an adversarial distortion δ^* in the wavelet domain:

$$\delta^* = \arg \min_{\delta} f(\text{IWT}(\text{WT}(x) + \delta)) + \lambda \|\delta\|_0, \quad (7)$$

where x is a given image/audio signal, f is the Carlini-Wagner loss function (Chen et al., 2017), WT is the chosen (discrete or continuous) wavelet transform, and IWT is the corresponding inverse wavelet transform. As wavelet transforms extract the important features of the data, we expect the gradients of this new loss function to be even sparser than those of the corresponding pixel-domain loss function (Cai et al., 2020b, Figure 1). Moreover, the inverse wavelet transform spreads the energy of the sparse perturbation, resulting in more natural-seeming attacked signals,

as compared with pixel-domain sparse attacks (Cai et al., 2020b, Figure 6).

1. **Sparse DWT attacks.** The discrete wavelet transform (DWT) is a well-known method for data compression and denoising (Mallat, 1999; Cai et al., 2012). Many real-world media data are compressed and stored in the form of DWT coefficients (e.g. JPEG-2000 for images and Dirac for videos), thus attacking the wavelet domain is more direct in these cases. Since DWT does not increase the problem dimension⁴, the query complexity of sparse wavelet-domain attacks is the same as sparse pixel-domain attacks. An interesting variation is to only attack the important (i.e. large) wavelet coefficients. We explore this further in Section 5. This can reduce the attack problem dimension by 60%–80% for typical image datasets. Nevertheless, for large, modern color images, this dimension can still be massive.
2. **Sparse CWT attacks.** For oscillatory signals, the continuous wavelet transform (CWT) with analytic wavelets is preferred (Mallat, 1999; Lilly & Olhede, 2010). Unlike DWT, the dimension of the CWT coefficients is much larger than the original signal dimension. For example, attacking even 1 second audio clips in a CWT domain results in a problem of size $d > 1.7$ million (see Section 5.3)!

The idea of adversarial attacks on DWT coefficients was also proposed in (Anshumaan et al., 2020), but they assume a white-box model and study only dense attacks on discrete Haar wavelets. (Din et al., 2020) considers a “steganographic” attack, where the important wavelet coefficients of a target image are “hidden” within the wavelet transform coefficients of a victim image. We appear to be the first to connect (both discrete and continuous) wavelet transforms to sparse zeroth-order adversarial attacks.

5. Empirical results

In this section, we first show the empirical advantages of ZO-BCD with synthetic examples. Then, we demonstrate the performance of ZO-BCD in two real-world applications: (i) sparse DWT attacks on images, and (ii) sparse CWT attacks on audio signals. We compare the two versions of ZO-BCD (see Algorithm 2) against two venerable zeroth-order algorithms—FDSA (Kiefer et al., 1952) and SPSA⁵ (Spall, 1998)—as well as three more recent contributions: ZO-SCD (Chen et al., 2017), ZORO (Cai et al., 2020b) and

⁴When using periodic boundary extension. If another boundary extension is used, the dimension of the wavelet coefficients may increase slightly, depending on the size of the filters and the level of the transform.

⁵SPSA using Rademacher sample directions coincides with Random Search (Nesterov & Spokoiny, 2017).

LM-MA-ES (Loshchilov et al., 2018). ZO-SCD is a zeroth-order (non-block) coordinate descent method. ZORO uses a similar gradient estimator as ZO-BCD, but computes the full gradient. LM-MA-ES is a recently proposed extension of CMA-ES (Hansen & Ostermeier, 2001) to the large-scale setting. In Section 5.2, we consider ZO-SGD (Ghadimi & Lan, 2013), a variance-reduced version of SPSA, as this has empirically shown better performance on this task than SPSA⁶. We also consider ZO-AdaMM (Chen et al., 2019), a zeroth-order method incorporating momentum. The experiments in Section 5.1 were executed from Matlab 2020b on a laptop with Intel i7-8750H CPU and 32GB RAM. The experiments in Sections 5.2 and 5.3 were executed on a workstation with Intel i9-9940X CPU, 128GB RAM, and two of Nvidia RTX-3080 GPUs. All code is available online at <https://github.com/YuchenLou/ZO-BCD>.

5.1. Synthetic examples

We study the performance of ZO-BCD with noisy oracles on the zeroth-order optimization problem $\min_{x \in \mathbb{R}^d} f(x)$ for two selected objective functions:

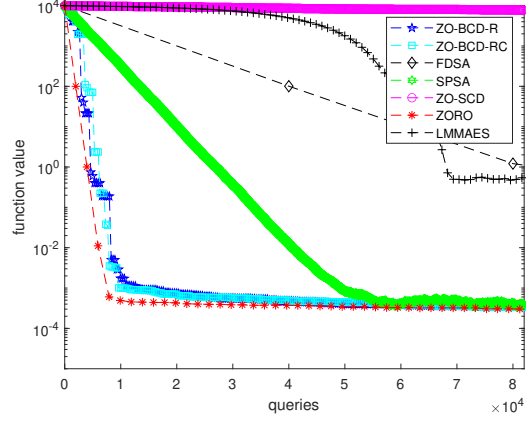
- (a) Sparse quadratic function: $f(x) = \frac{1}{2}x^T Ax$, where A is a diagonal matrix with s non-zero entries.
- (b) Max- s -sum-squared function: $f(x) = \frac{1}{2} \sum_{m_i}^s x_{m_i}^2$, where x_{m_i} is the i -th largest-in-magnitude entry of x . This problem is more complicated than (a) as m_i changes with x .

We use $d = 20,000$ and $s = 200$ in both problems, so they have high ambient dimension with sparse gradients.

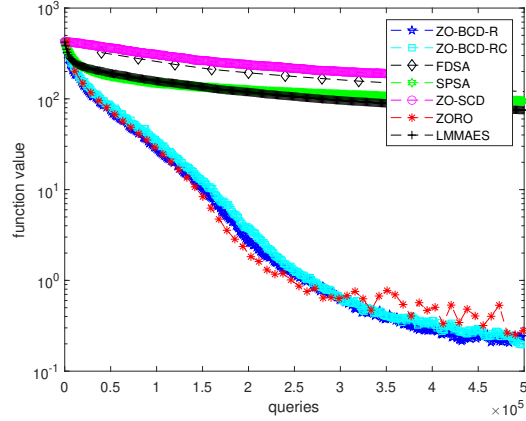
As can be seen in Figure 2, both versions of ZO-BCD effectively exploit the gradient sparsity, and have very competitive performance in terms of queries. In particular, ZO-BCD converges more stably than the state-of-the-art ZORO in max- s -sum-squared problem while its computational and memory complexities are much lower. SPSA’s query efficiency is roughly the same as that of ZO-BCD and ZORO when the gradient support does not change (see Figure 2a); however, it is *significantly worse* when the gradient support is allowed to change (see Figure 2b).

Number of blocks. We study the performance of ZO-BCD with different numbers of blocks. The numerical results are summarized in Figure 3 and Table 1. Note that the runtime of each query can vary a lot between problems, so we only count the empirical runtime excluding the time of making queries. As a rule of thumb, we find that using fewer blocks yields smoother convergence and a more accurate final solution (see Figure 3), at the cost of a higher runtime

⁶Variance reduction did little to improve the performance of SPSA in the experiments of Section 5.1.



(a) Sparse Quadratic.

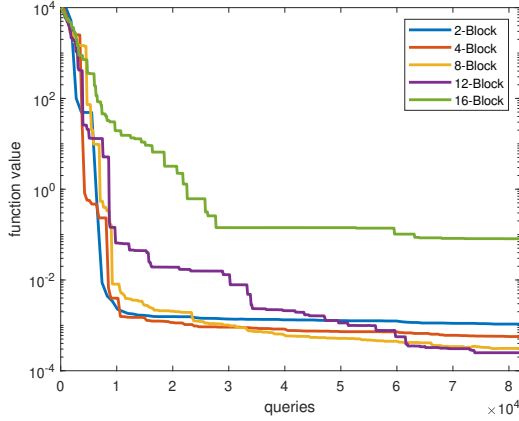


(b) Max- s -sum-squared.

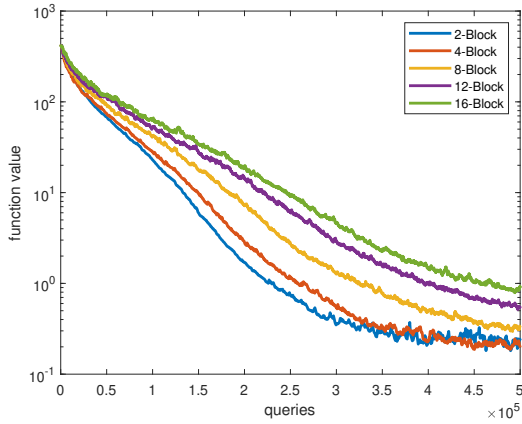
Figure 2. Function values v.s. queries for ZO-BCD (-R and -RC, with 5 blocks) and four other representative zeroth-order methods. ZO-BCD is fast and stable (while running faster with less memory).

(see Table 1). This phenomenon matches our theoretical result in Theorem 3.1. Generally speaking, we recommend using a mild number of blocks to balance between ZO-BCD’s speed and convergence performance.

Scalability. Since ZO-BCD and ZORO are the only methods that have competitive convergences in the query complexity experiments (see Figure 2), it is only meaningful to compare their computational complexities with respect to problem dimension d . We record the runtime of ZO-BCD and ZORO for solving the sparse quadratic function with varying problem dimensions, where the stopping condition is set to be $f(x_k) \leq 10^{-2}$ for all tests. Similar to the experiment of varying numbers of blocks, we only count the empirical runtime excluding the time of making queries in this experiment. The test results are presented in Figure 4, where one can see that ZO-BCD-R has significant speed advantage over ZORO and ZO-BCD-RC is even faster, es-



(a) Sparse Quadric.



(b) Max-s-squared-sum.

Figure 3. Function values v.s. queries for ZO-BCD-R with different numbers of blocks.

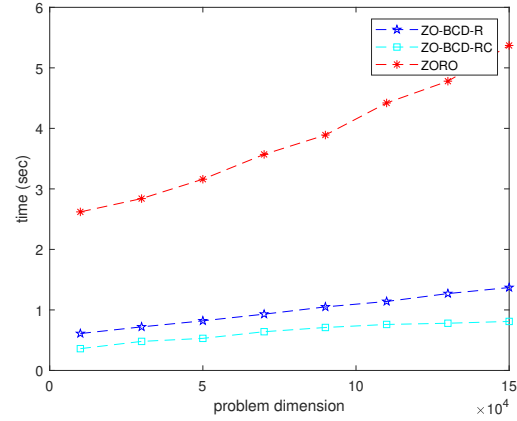
pecially when problem dimension is large.

5.2. Sparse DWT attacks on images

We consider a wavelet domain, untargeted, per-image attack on the ImageNet dataset (Deng et al., 2009) with the pre-trained Inception-v3 model (Szegedy et al., 2016), as discussed in Section 4. We use the famous ‘db45’ wavelet (Daubechies, 1992) with 3-level DWT in these attacks. Empirical performance was evaluated by attacking 1000 randomly selected ImageNet pictures that were initially classified correctly. In addition to full wavelet domain attacks using both ZO-BCD-R and ZO-BCD-RC, we experiment with only attacking large wavelet coefficients, *i.e.* the important components of the images in terms of the wavelet basis. If we only attack wavelet coefficients greater than 0.05 in magnitude, the problem dimension is reduced by an average of 67.3% for the tested images; nevertheless, the attack problem dimension is still as large as $\sim 90,000$, so

 Table 1. Runtime per iteration and the number of iterations to reach tolerance for ZO-BCD-R with varying number of blocks (J).

J	Sparse Quadric.		Max- s -squared-sum	
	SEC/ITR	ITR to 10^{-2}	SEC/ITR	ITR to 10^0
2	.1093 sec	8	.1362 sec	249
4	.0244 sec	20	.0358 sec	605
8	.0054 sec	45	.0116 sec	1651
12	.0026 sec	224	.0054 sec	3185
16	.0019 sec	N/A	.0042 sec	5090



(a) Sparse Quadric.

Figure 4. Runtime v.s. problem dimension for ZO-BCD (-R and -RC, with 5 blocks) and ZORO.

ZO-BCD is still suitable for this attack problem.

The test results are summarized in Table 2. All three versions of ZO-BCD wavelet attack beat the other state-of-the-art methods in both attack success rate and ℓ_2 distortion, and the large-coefficients-only (*i.e.* we only attack wavelet coefficients with $\text{abs} \geq 0.05$) wavelet attack by ZO-BCD-R achieves the best results. Furthermore, ZO-BCD is robust to the choice of the number of blocks and sparsity, as summarized in Table 3. We present a few visual examples of these adversarial attacks in Figure 5. More examples, and detailed experimental settings, can be found in Appendix D.

5.3. Sparse CWT attacks on audio signals

We consider targeted per-clip audio adversarial attacks on the SpeechCommands dataset (Warden, 2018), which consists of 1-second audio clips, each containing a one-word voice command, *e.g.* “yes” or “left”. The audio sampling rate is 16kHz thus each clip is a 16,000 real valued vector. Adversarial attacks against this data set have been considered in (Alzantot et al., 2018; Vadillo & Santana, 2019; Li et al., 2020) and (Xie et al., 2020), although with the ex-

Table 2. Results of untargeted adversarial attack on images using various zeroth-order algorithms. Attack success rate (ASR), average final ℓ_2 distortion (on pixel domain), average iterations and number of queries till 1st successful attack. ZO-BCD-R(large coeff.) stands for applying ZO-BCD-R to attack only large wavelet coefficients (abs ≥ 0.05).

METHODS	ASR	ℓ_2 DIST	QUERIES
ZO-SCD	78%	57.5	2400
ZO-SGD	78%	37.9	1590
ZO-AdaMM	81%	28.2	1720
ZORO	90%	21.1	2950
ZO-BCD-R	92%	14.1	2131
ZO-BCD-RC	92%	14.2	2090
ZO-BCD-R(large coeff.)	96%	13.7	1662

Table 3. Results of sparse DWT adversarial attack on images, using ZO-BCD-R and different values of s (sparsity) and J (number of blocks). ZO-BCD-R is robust to the particular choice of s and J . Note $d = 676$, 353 is the problem dimension.

ZO-BCD-R	ASR	ℓ_2 DIST	QUERIES
$J = 2000, s = 0.05d$	93%	15.3	2423
$J = 4000, s = 0.05d$	91%	14.0	2109
$J = 8000, s = 0.05d$	93%	14.8	2145
$J = 12000, s = 0.05d$	90%	13.8	1979
$J = 4000, s = 0.01d$	86%	22.6	2440
$J = 4000, s = 0.025d$	93%	16.5	2086
$J = 4000, s = 0.1d$	90%	13.1	2364

ception of (Alzantot et al., 2018) all these works consider a white-box setting⁷. The victim model is a pre-trained, 5 layer, convolutional network called `commandNet` (The MathWorks Inc., 2020). The architecture is essentially as proposed in (Sainath & Parada, 2015). It takes as input the bark spectrum coefficients of a given audio clip, a transform closely related to the Mel Frequency transform. The test classification accuracy of this model (on un-attacked audio clips) is 94.46%. We use the Morse (Olhede & Walden, 2002) continuous wavelet transform with 111 frequencies, resulting in a problem dimension of $111 \times 16,000 = 1,776,000$. As discussed in (Carlini & Wagner, 2018), the appropriate measure of size for the attacking distortion δ is relative loudness:

$$\text{dB}_x(\delta) := 20 \left(\max_i \log_{10}(|x_i|) - \max_i \log_{10}(|\delta_i|) \right).$$

The results are detailed in Table 5 and Figure 6. Overall, we achieve a 97.93% ASR using a mean of 7073 queries. Our attacking distortions have a mean volume of -6.32dB . As can be seen, our proposed attack exceeds the state of the

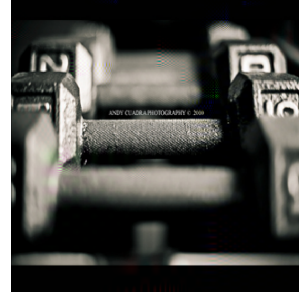
⁷There are other subtle differences in the threat models considered in these works, as compared to ours (see Appendix E.2).



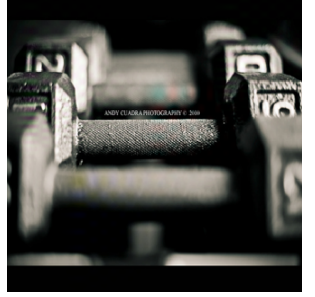
(a) ZO-BCD-R: “barbershop” → “flagpole”



(b) ZO-BCD-R (large coeff.): “barbershop” → “flagpole”



(c) ZO-BCD-R: “dumbbell” → “computer keyboard”



(d) ZO-BCD-R (large coeff.): “dumbbell” → “computer keyboard”

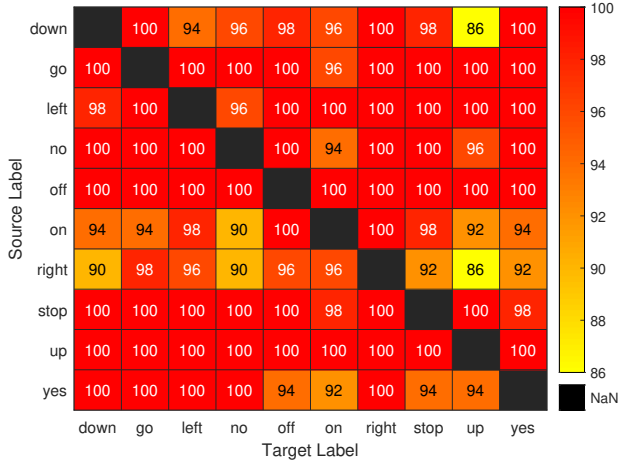
Figure 5. Examples of wavelet attacked images by ZO-BCD-R and ZO-BCD-R (attack restricted to only large coefficients, i.e. abs ≥ 0.05), true labels and mis-classified labels.

art in attack success rate (ASR), surpassing even white-box attacks! This is not to claim that our proposed method is strictly better than others, as there are multiple factors to consider when judging the “goodness” of an attack (ASR, attack distortion, universality etc.), see Appendix E.2. The attacking noise can be heard as a slight “hiss”, or white noise in the attacked audio clips. The original keyword however is easy for a human listener to make out. We encourage the reader to listen to a few examples, available at <https://github.com/YuchenLou/ZO-BCD>.

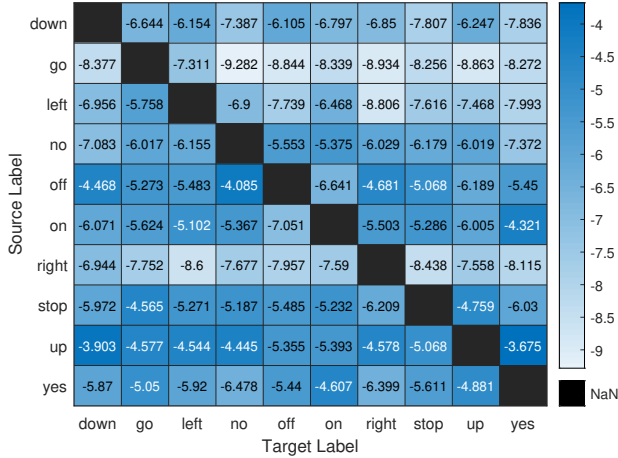
Out of curiosity, we also tested using ZO-BCD to craft untargeted adversarial attacks in the time domain (i.e. without using a wavelet transform) for 1000 randomly selected audio clips. The results are underwhelming; indeed the attacking perturbation is on average significantly *louder* than the victim audio clip (see Table 4)! This suggests attacking in a wavelet domain is much more effective than attacking in the original signal domain.

6. Conclusion

We have introduced ZO-BCD, a novel zeroth-order optimization algorithm. ZO-BCD enjoys strong, albeit probabilistic, convergence guarantees. We have also introduced a new



(a) Attack success rate.



(b) Relative loudness, in decibels.

Figure 6. Detailed results for *targeted* sparse wavelet attacks on audio signals.

paradigm in adversarial attacks on classifiers: the sparse wavelet domain attack. On medium-scale test problems the performance of ZO-BCD matches or exceeds that of state-of-the-art zeroth order optimization algorithms, as predicted by theory. However, the low per-iteration computational and memory requirements of ZO-BCD means that it can tackle huge-scale problems that for other zeroth-order algorithms are intractable. We demonstrate this by successfully using ZO-BCD to craft adversarial examples, to both image and audio classifiers, in wavelet domains where the problem size can exceed 1.7 million.

Acknowledgements

We thank Zhenliang Zhang for valuable discussions on the related work. We also thank the anonymous reviewers for their helpful feedback.

Table 4. Results for *untargeted* attacks on audio signals using ZO-BCD-R in the time domain, in the wavelet domain using a step-size of 0.02 and in the wavelet domain using a step-size of 0.05. Attack success rate (ASR), average final Decibel distortion and average number of queries to 1st successful attack.

DOMAINS	ASR	dB DIST	QUERIES
Time	100%	+1.5597	894
Wavelet (0.02)	99.9%	-13.8939	3452
Wavelet (0.05)	100%	-7.1192	2502

Table 5. Results of attacks on *SpeechCommands* dataset. A = (Alzantot et al., 2018), V&S = (Vadillo & Santana, 2019), Li = (Li et al., 2020), Xie = (Xie et al., 2020). Univ. = Universal.

METHOD	ASR	UNIV.	BLACK-BOX	TARGETED
A.	89.0%	NO	YES	YES
V&S	70.4%	YES	NO	NO
Li	96.8%	YES	NO	YES
Xie	97.8%	NO	NO	YES
ZO-BCD	97.9%	NO	YES	YES

References

- Alzantot, M., Balaji, B., and Srivastava, M. Did you hear that? Adversarial examples against automatic speech recognition. *arXiv preprint arXiv:1801.00554*, 2018.
- Alzantot, M., Sharma, Y., Chakraborty, S., Zhang, H., Hsieh, C.-J., and Srivastava, M. B. Genattack: Practical black-box attacks with gradient-free optimization. In *Proceedings of the Genetic and Evolutionary Computation Conference*, pp. 1111–1119, 2019.
- Anshumaan, D., Agarwal, A., Vatsa, M., and Singh, R. Wavetransform: Crafting adversarial examples via input decomposition. In *Computer Vision – ECCV 2020 Workshops*, pp. 152–168. Springer, Cham, 2020.
- Balasubramanian, K. and Ghadimi, S. Zeroth-order (non)-convex stochastic optimization via conditional gradient and gradient updates. In *Advances in Neural Information Processing Systems*, pp. 3455–3464, 2018.
- Berahas, A. S., Cao, L., Choromanski, K., and Scheinberg, K. A theoretical and empirical comparison of gradient approximations in derivative-free optimization. *arXiv preprint arXiv:1905.01332*, 2019.
- Bergstra, J. and Bengio, Y. Random search for hyperparameter optimization. *Journal of machine learning research*, 13(2), 2012.
- Bertsekas, D. P. Nonlinear programming. *Journal of the Operational Research Society*, 48(3):334–334, 1997.

- Cai, H., McKenzie, D., Yin, W., and Zhang, Z. A one-bit, comparison-based gradient estimator. *arXiv preprint arXiv:2010.02479*, 2020a.
- Cai, H., McKenzie, D., Yin, W., and Zhang, Z. Zeroth-order regularized optimization (ZORO): Approximately sparse gradients and adaptive sampling. *arXiv preprint arXiv:2003.13001*, 2020b.
- Cai, J.-F., Dong, B., Osher, S., and Shen, Z. Image restoration: Total variation, wavelet frames, and beyond. *Journal of the American Mathematical Society*, 25(4):1033–1089, 2012.
- Carlini, N. and Wagner, D. Audio adversarial examples: Targeted attacks on speech-to-text. In *2018 IEEE Security and Privacy Workshops*, pp. 1–7. IEEE, 2018.
- Chen, P.-Y., Zhang, H., Sharma, Y., Yi, J., and Hsieh, C.-J. ZOO: Zeroth order optimization based black-box attacks to deep neural networks without training substitute models. In *Proceedings of the 10th ACM workshop on artificial intelligence and security*, pp. 15–26, 2017.
- Chen, X., Liu, S., Xu, K., Li, X., Lin, X., Hong, M., and Cox, D. Zo-AdaMM: Zeroth-order adaptive momentum method for black-box optimization. In *Advances in Neural Information Processing Systems*, pp. 7204–7215, 2019.
- Choromanski, K., Pacchiano, A., Parker-Holder, J., Tang, Y., Jain, D., Yang, Y., Iscen, A., Hsu, J., and Sindhvani, V. Provably robust blackbox optimization for reinforcement learning. In *Conference on Robot Learning*, pp. 683–696. PMLR, 2020.
- Cisse, M., Adi, Y., Neverova, N., and Keshet, J. Houdini: Fooling deep structured visual and speech recognition models with adversarial examples. In *Proceedings of the 31st International Conference on Neural Information Processing Systems*, pp. 6980–6990, 2017.
- Daubechies, I. *Ten lectures on wavelets*. SIAM, 1992.
- Deng, J., Dong, W., Socher, R., Li, L.-J., Li, K., and Fei-Fei, L. Imagenet: A large-scale hierarchical image database. In *2009 IEEE conference on computer vision and pattern recognition*, pp. 248–255. Ieee, 2009.
- Din, S. U., Akhtar, N., Younis, S., Shafait, F., Mansoor, A., and Shafique, M. Steganographic universal adversarial perturbations. *Pattern Recognition Letters*, 135:146–152, 2020.
- Flaxman, A. D., Kalai, A. T., and McMahan, H. B. Online convex optimization in the bandit setting: Gradient descent without a gradient. In *Proceedings of the sixteenth annual ACM-SIAM symposium on Discrete algorithms*, pp. 385–394, 2005.
- Ghadimi, S. and Lan, G. Stochastic first-and zeroth-order methods for nonconvex stochastic programming. *SIAM Journal on Optimization*, 23(4):2341–2368, 2013.
- Golovin, D., Karro, J., Kochanski, G., Lee, C., Song, X., and Zhang, Q. Gradientless descent: High-dimensional zeroth-order optimization. In *International Conference on Learning Representations*, 2019.
- Hannun, A., Case, C., Casper, J., Catanzaro, B., Diamos, G., Elsen, E., Prenger, R., Satheesh, S., Sengupta, S., Coates, A., et al. Deep speech: Scaling up end-to-end speech recognition. *arXiv preprint arXiv:1412.5567*, 2014.
- Hansen, N. and Ostermeier, A. Completely derandomized self-adaptation in evolution strategies. *Evolutionary computation*, 9(2):159–195, 2001.
- Huang, M., Pang, Y., and Xu, Z. Improved bounds for the RIP of subsampled circulant matrices. *arXiv preprint arXiv:1808.07333*, 2018.
- Hutter, F., Hoos, H., and Leyton-Brown, K. An efficient approach for assessing hyperparameter importance. In *International conference on machine learning*, pp. 754–762. PMLR, 2014.
- Ilyas, A., Engstrom, L., Athalye, A., and Lin, J. Black-box adversarial attacks with limited queries and information. In *International Conference on Machine Learning*, pp. 2137–2146. PMLR, 2018.
- Jamieson, K. G., Nowak, R. D., and Recht, B. Query complexity of derivative-free optimization. In *Proceedings of the 25th International Conference on Neural Information Processing Systems-Volume 2*, pp. 2672–2680, 2012.
- Kiefer, J., Wolfowitz, J., et al. Stochastic estimation of the maximum of a regression function. *The Annals of Mathematical Statistics*, 23(3):462–466, 1952.
- Krahmer, F., Mendelson, S., and Rauhut, H. Suprema of chaos processes and the restricted isometry property. *Communications on Pure and Applied Mathematics*, 67(11):1877–1904, 2014.
- Li, Z., Wu, Y., Liu, J., Chen, Y., and Yuan, B. Advpulse: Universal, synchronization-free, and targeted audio adversarial attacks via subsecond perturbations. In *Proceedings of the 2020 ACM SIGSAC Conference on Computer and Communications Security*, pp. 1121–1134, 2020.
- Lian, X., Zhang, H., Hsieh, C.-J., Huang, Y., and Liu, J. A comprehensive linear speedup analysis for asynchronous stochastic parallel optimization from zeroth-order to first-order. *arXiv preprint arXiv:1606.00498*, 2016.

- Lilly, J. M. and Olhede, S. C. On the analytic wavelet transform. *IEEE transactions on information theory*, 56(8):4135–4156, 2010.
- Loshchilov, I., Glasmachers, T., and Beyer, H.-G. Large scale black-box optimization by limited-memory matrix adaptation. *IEEE Transactions on Evolutionary Computation*, 23(2):353–358, 2018.
- Mallat, S. *A wavelet tour of signal processing*. Elsevier, 1999.
- Mania, H., Guy, A., and Recht, B. Simple random search of static linear policies is competitive for reinforcement learning. In *Proceedings of the 32nd International Conference on Neural Information Processing Systems*, pp. 1805–1814, 2018.
- Mendelson, S., Rauhut, H., Ward, R., et al. Improved bounds for sparse recovery from subsampled random convolutions. *The Annals of Applied Probability*, 28(6):3491–3527, 2018.
- Modas, A., Moosavi-Dezfooli, S.-M., and Frossard, P. Sparsefool: A few pixels make a big difference. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, pp. 9087–9096, 2019.
- Needell, D. and Tropp, J. A. Cosamp: Iterative signal recovery from incomplete and inaccurate samples. *Applied and computational harmonic analysis*, 26(3):301–321, 2009.
- Nesterov, Y. Efficiency of coordinate descent methods on huge-scale optimization problems. *SIAM Journal on Optimization*, 22(2):341–362, 2012.
- Nesterov, Y. and Spokoiny, V. Random gradient-free minimization of convex functions. *Foundations of Computational Mathematics*, 17(2):527–566, 2017.
- Olhede, S. C. and Walden, A. T. Generalized Morse wavelets. *IEEE Transactions on Signal Processing*, 50(11):2661–2670, 2002.
- Reeja-Jayan, B., Harrison, K. L., Yang, K., Wang, C.-L., Yilmaz, A., and Manthiram, A. Microwave-assisted low-temperature growth of thin films in solution. *Scientific reports*, 2(1):1–8, 2012.
- Sainath, T. N. and Parada, C. Convolutional neural networks for small-footprint keyword spotting. In *Sixteenth Annual Conference of the International Speech Communication Association*, 2015.
- Salimans, T., Ho, J., Chen, X., Sidor, S., and Sutskever, I. Evolution strategies as a scalable alternative to reinforcement learning. *arXiv preprint arXiv:1703.03864*, 2017.
- Spall, J. C. An overview of the simultaneous perturbation method for efficient optimization. *Johns Hopkins apl technical digest*, 19(4):482–492, 1998.
- Szegedy, C., Vanhoucke, V., Ioffe, S., Shlens, J., and Wojna, Z. Rethinking the inception architecture for computer vision. In *Proceedings of the IEEE conference on computer vision and pattern recognition*, pp. 2818–2826, 2016.
- Taori, R., Kamsetty, A., Chu, B., and Vemuri, N. Targeted adversarial examples for black box audio systems. In *2019 IEEE Security and Privacy Workshops (SPW)*, pp. 15–20. IEEE, 2019.
- Tappenden, R., Richtárik, P., and Gondzio, J. Inexact coordinate descent: Complexity and preconditioning. *Journal of Optimization Theory and Applications*, 170(1):144–176, 2016.
- The MathWorks Inc. Speech command recognition using deep learning. <https://www.mathworks.com/help/deeplearning/ug/deep-learning-speech-recognition.html>, 2020. Accessed: 2021-01-15.
- Vadillo, J. and Santana, R. Universal adversarial examples in speech command classification. *arXiv preprint arXiv:1911.10182*, 2019.
- Wang, Y., Du, S., Balakrishnan, S., and Singh, A. Stochastic zeroth-order optimization in high dimensions. In *International Conference on Artificial Intelligence and Statistics*, pp. 1356–1365, 2018.
- Warden, P. Speech commands: A dataset for limited-vocabulary speech recognition. *arXiv preprint arXiv:1804.03209*, 2018.
- Xie, Y., Li, Z., Shi, C., Liu, J., Chen, Y., and Yuan, B. Enabling fast and universal audio adversarial attack using generative model. *arXiv preprint arXiv:2004.12261*, 2020.

A. Proofs for Section 2

Proof of Theorem 2.1. Consider the function $f^{(j)}(t) = f(x + U^{(j)}t)$. This function has gradient $g^{(j)}$ and Hessian $\nabla_{jj}^2 f$. By Assumption 5 $\|g^{(j)}\|_0 \leq \|g\|_0 \leq s$ while $\|\nabla_{jj}^2 f\|_1 \leq H$ from Assumption 6. Thus, $f^{(j)}(t)$ satisfies the assumptions of Corollary 2.7 of (Cai et al., 2020b) so Theorem 2.1 follows from this result. $\square$

Proof of Theorem 2.2. For notational convenience, for this proof only we let $s := s_{\text{exact}}$. Let $g_{i_1}, \dots, g_{i_k}, \dots, g_{i_s}$ denote the non-zero entries of $g := g(x)$, after the permutation has been applied. Let Y_j be the random variable counting the number of g_{i_k} within block j :

$$Y_j = \#\{i_k : i_k \text{ in block } j\}.$$

Thus, the random vector $\mathbf{Y} = (Y_1, \dots, Y_J) \in \mathbb{R}^J$ obeys the multinomial distribution. Observe $\sum_{j=1}^J Y_j = s$ and, because the blocks are equally sized, $\mathbb{E}(Y_j) = \frac{s}{J}$.

By Chernoff's bound, for $\Delta > 0$ and each Y_j individually:

$$\mathbb{P}\left[\left|Y_j - \frac{s}{J}\right| \geq \Delta \frac{s}{J}\right] \leq 2e^{-\frac{\Delta^2 \mathbb{E}(Y_j)}{3}} = 2e^{-\frac{\Delta^2 s}{3J}}.$$

Applying the union bound:

$$\begin{aligned} & \mathbb{P}\left[\exists j \text{ s.t. } \left|Y_j - \frac{s}{J}\right| \geq \Delta \frac{s}{J}\right] \\ & \leq \sum_{j=1}^J \mathbb{P}\left[\left|Y_j - \frac{s}{J}\right| \geq \Delta \frac{s}{J}\right] \leq 2Je^{-\frac{\Delta^2 s}{3J}}, \end{aligned}$$

and so:

$$\begin{aligned} & \mathbb{P}\left[\left|Y_j - \frac{s}{J}\right| \leq \Delta \frac{s}{J}, \forall j \in [1, \dots, J]\right] \\ & = 1 - \mathbb{P}\left[\exists j \text{ s.t. } \left|Y_j - \frac{s}{J}\right| \geq \Delta \frac{s}{J}\right] \\ & \geq 1 - 2Je^{-\frac{\Delta^2 s}{3J}}. \end{aligned}$$

This finishes the proof. $\square$

Proof of Corollary 2.3. From Theorem 2.2 we have that, with probability at least $1 - 2J \exp(\frac{-0.01s_{\text{exact}}}{3J})$, $\|g^{(j)}\|_0 \leq 1.1s_{\text{exact}}/J \leq s$. Assuming this is true, (5) holds with probability $1 - (s/d)^{b_2 s}$. These events are independent, thus the probability that they both occur is:

$$\left(1 - 2J \exp\left(\frac{-0.01s_{\text{exact}}}{3J}\right)\right) (1 - (s/d)^{b_2 s})$$

Because $s \ll d$ the term exponential in s_{exact}/J is significantly larger than $(s/d)^{b_2 s}$. Expanding, and keeping only dominant terms we see that this probability is equal to $1 - \mathcal{O}\left(J \exp(\frac{-0.01s_{\text{exact}}}{3J})\right)$. $\square$

We emphasize that Corollary 2.3 holds for all j . Before proceeding, we remind the reader that for fixed s the *restricted isometry constant* of $Z \in \mathbb{R}^{m \times n}$ is defined as the smallest $\delta > 0$ such that:

$$(1 - \delta)\|v\|_2^2 \leq \|Zv\|_2^2 \leq (1 + \delta)\|v\|_2^2$$

for all $v \in \mathbb{R}^n$ satisfying $\|v\|_0 \leq s$. The key ingredient to the proof of Theorem 2.4 is the following result:

Theorem A.1 ((Krahmer et al., 2014, Theorem 1.1)). *Let $z \in \mathbb{R}^n$ be a Rademacher random vector and choose a random subset $\Omega = \{j_1, \dots, j_m\} \subset \{1, \dots, n\}$ of cardinality $m = c\delta^{-2}s \log^2(s) \log^2(n)$. Let $Z \in \mathbb{R}^{m \times n}$ denote the matrix with rows $\frac{1}{\sqrt{m}}C_{j_i}(z)$. Then $\delta_s(Z) < \delta$ with probability $1 - n^{-\log(n) \log^2(s)}$.*

c is a universal constant, independent of s, n and δ . Similar results may be found in (Mendelson et al., 2018) and (Huang et al., 2018). In (Krahmer et al., 2014) a more general version of this theorem is provided, which allows the entries of z to be drawn from any sub-Gaussian distribution.

Proof of Theorem 2.4. Let Z be the sensing matrix with rows $\frac{1}{\sqrt{m}}C_{j_i}(z)$. By appealing to Theorem A.1 with $s = 4s$, $n = d/J$ and $\delta = 0.3843$ we get $\delta_{4s}(Z) \leq 0.3843$, with probability $1 - (d/J)^{\log(d/J) \log^2(4s)}$. From Theorem 2.2 we have that, with probability at least $1 - 2J \exp(\frac{-0.01s_{\text{exact}}}{3J})$, $\|g^{(j)}\|_0 \leq 1.1s_{\text{exact}}/J \leq s$. Assuming $\delta_{4s}(Z) \leq 0.3843$, Theorem 2.4 follows by the same proof as in (Cai et al., 2020b, Corollary 2.7). The events $\|g^{(j)}\|_0 \leq s/J$ and $\delta_{4s}(Z) \leq 0.3843$ are independent, hence they both occur with probability:

$$\begin{aligned} & \left(1 - 2J \exp\left(\frac{-0.01s_{\text{exact}}}{3J}\right)\right) \left(1 - (d/J)^{\log(d/J) \log^2(4s)}\right) \\ & \geq 1 - 2J \exp\left(\frac{-0.01s_{\text{exact}}}{3J}\right) - (d/J)^{\log(d/J) \log^2(4s)}. \end{aligned} \tag{8}$$

Note that the third term in the right side of (8) is *universal*, i.e. it holds for all $x \in \mathbb{R}^d$, as it depends only on the choice of Z , not the choice of x . $\square$

B. ZO-BCD for unequally-sized blocks

Using randomly assigned, equally-sized blocks is an important part of the ZO-BCD framework as it allows one to consider a block sparsity $\approx s/J$, instead of the worst case sparsity of s . Nevertheless, there may be situations where it is preferable to use user-defined, unequally-sized blocks. For such cases we recommend the following (we discuss the modifications here for ZO-BCD-R, but with obvious changes it also applies to ZO-BCD-RC). Let $s^{(j)} \leq s$ be an upper estimate of the sparsity of the j -th block gradient: $\|g^{(j)}(x)\|_0 \leq s^{(j)}$. Let

$m^{(j)} = b_1 s^{(j)} \log(D/J)$ (and use the analogous formula for ZO-BCD-RC) and define $m^{\max} = \max_j m^{(j)}$. When initializing ZO-BCD-R, generate $m^{\max}$ Rademacher random variables: $z_1, \dots, z_{m^{\max}} \in \mathbb{R}^{d^{\max}}$. At each iteration, if block j is selected, randomly select $i_1, \dots, i_{m^{(j)}}$ from $1, \dots, m^{\max}$ and for $k = 1, \dots, m^{(j)}$ let $\tilde{z}_{i_k}^{(j)} \in \mathbb{R}^{d^{(j)}}$ denote the vector formed by taking the first $d^{(j)}$ components of z_{i_k} . Use $\{\tilde{z}_{i_k}^{(j)}\}_{k=1}^{m^{(j)}}$ as the input to Algorithm 1. Note that the $\{\tilde{z}_{i_k}^{(j)}\}_{k=1}^{m^{(j)}}$ possess the same statistical properties as the $\{z_{i_k}\}_{k=1}^{m^{(j)}}$ (i.e. they are i.i.d. Rademacher vectors) so using them as sampling directions will result in the same bound on $\|g_k^{(j)} - \hat{g}_k^{(j)}\|_2$ as Corollary 2.3.

C. Proofs for Section 3

Our proof utilizes the main result of (Tappenden et al., 2016). This paper requires $\alpha_k = \frac{1}{L_{j_k}}$, i.e. the step size at the k -th iteration is inversely proportional to the block Lipschitz constant. This is certainly ideal, but impractical. In particular, if the blocks are randomly selected it seems implausible that one would have good estimates of the L_j . Of course, since $L_j \leq L_{\max}$ we observe that $L_{\max}$ is a Lipschitz constant for every block, and thus we may indeed take $\alpha_k = \alpha = \frac{1}{L_{\max}}$. This results in a slightly slower convergence resulted, reflected in a factor of $L_{\max}$ in Theorem 3.1. Throughout this section we shall assume f satisfies Assumptions 1–6. For all $j = 1, \dots, J$ define:

$$V_j(x, t) = \langle g^{(j)}(x), t \rangle + \frac{L_{\max}}{2} \|t\|_2^2,$$

so that, by Lipschitz differentiability:

$$f(x_k + U^{(j)}t) \leq f(x_k) + V_j(x_k, t).$$

Define $t_{k,j}^* := \arg \min V_j(x_k, t) := -\frac{1}{L_{\max}} g_k^{(j)}$ while let $t'_{k,j}$ be the update step taken by ZO-BCD, i.e. $t'_{k,j} = \frac{1}{L_{\max}} \hat{g}_k^{(j)}$.

Lemma C.1. Suppose f satisfies Assumptions 1–3. Then $f(x) - f^* \geq \frac{1}{2L_{\max}} \|g^{(j)}(x)\|_2^2$ for any $x \in \mathbb{R}^d$ and any $j = 1, \dots, J$.

Proof. Define $h_x : \mathbb{R}^{d^{(j)}} \rightarrow \mathbb{R}$ as $h_x(t) := f(x + U^{(j)}t)$ where $U^{(j)}$ is as described in Section 1. Since $U^{(j)}$ is a linear transformation and f is convex, h_x is also convex. By Assumption 1 and $L_j \leq L_{\max}$, h_x is $L_{\max}$ -Lipschitz differentiable. From Assumption 3 it follows $\mathcal{Y}^* = \arg \min_t h_x(t)$ is non-empty, and $h_x^* := \min_t h_x(t) \geq f^*$. Thus, from (Bertsekas, 1997, Proposition B.3, part (c.ii)), we have:

$$h_x(t) - h_x^* \geq \frac{1}{2L_{\max}} \|\nabla h_x(t)\|_2^2 = \frac{1}{2L_{\max}} \|g^{(j)}(x + U^{(j)}t)\|_2^2$$

for all t . Choose $t = 0$, and use $h_x(0) = f(x)$ and $f^* \leq h_x^*$ to obtain:

$$f(x) - f^* \geq h_x(0) - h_x^* \geq \frac{1}{2L_{\max}} \|g^{(j)}(x)\|_2^2.$$

This finishes the proof. $\square$

Lemma C.2. Let $\eta = 2\rho^{2n}$ and $\theta = \frac{4\tau^2\sigma H}{L_{\max}}$. For each iteration of ZO-BCD

$$V_j(x_k, t') - V_j(x_k, t^*) \leq \eta(f(x_k) - f^*) + \theta. \quad (9)$$

with probability $1 - \mathcal{O}\left(J \exp\left(-\frac{0.01s_{\text{exact}}}{3J}\right)\right)$ for ZO-BCD-R and with probability greater than

$$1 - 2J \exp\left(\frac{-0.01s_{\text{exact}}}{3J}\right) - (d/J)^{\log(d/J) \log^2(4.4s/J)}$$

for ZO-BCD-RC.

Proof. For notational convenience, we define $t^* := t_{k,j}^*$, $t' := t'_{k,j}$ and $e_k^{(j)} := \hat{g}_k^{(j)} - g_k^{(j)}$. By definition:

$$t'_{k,j} = -\frac{1}{L_{\max}} \hat{g}_k^{(j)} = -\frac{1}{L_{\max}} (g_k^{(j)} + e_k^{(j)}).$$

Moreover:

$$\begin{aligned} V_j(x_k, t^*) &= -\frac{1}{2L_{\max}} \|g_k^{(j)}\|_2^2 \\ V_j(x_k, t') &= -\frac{1}{2L_{\max}} \|g_k^{(j)}\|_2^2 + \frac{1}{2L_{\max}} \|e_k^{(j)}\|_2^2, \end{aligned}$$

and hence:

$$V_j(x_k, t') - V_j(x_k, t^*) \leq \frac{1}{2L_{\max}} \|e_k^{(j)}\|_2^2. \quad (10)$$

Recall $\hat{g}_k^{(j)}$ is the output of GradientEstimation with gradient sparsity level $s_{\text{block}} = 1.1s/J$. Using the error bound (5) and replacing s with $1.1s/J$:

$$\begin{aligned} \|e_k^{(j)}\|_2^2 &\leq \left(\rho^n \|g_k^{(j)}\|_2 + 2\tau\sqrt{\sigma H}\right)^2 \\ &\leq 2\rho^{2n} \|g_k^{(j)}\|_2^2 + 4\tau^2\sigma H. \end{aligned} \quad (11)$$

with the stated probabilities, as justified by Corollary 2.3 (for ZO-BCD-R) and Theorem 2.4 (for ZO-BCD-RC) Finally, from Lemma C.1 $\|g_k^{(j)}\|_2^2 \leq 2L_{\max} (f(x_k) - f^*)$ for any $j = 1, \dots, J$. Connecting this estimate with (10) and (11) we obtain:

$$V_j(x_k, t') - V_j(x_k, t^*) \leq \underbrace{2\rho^{2n}}_{=\eta} (f(x_k) - f^*) + \underbrace{\frac{4\tau^2\sigma H}{L_{\max}}}_{=\theta}.$$

This finishes the proof. $\square$

Proof of Theorem 3.1. Let p_j denote the probability that the j -th block is chosen for updating at the k -th iteration. Because ZO-BCD chooses blocks uniformly at random, $p_j = 1/J$ for all j . If (9) holds for all k then by (Tappenden et al., 2016, Theorem 6.1) if:

$$\begin{aligned} \eta^2 + \frac{4\theta}{c_1} &< 1 \text{ where } c_1 = 2JL_{\max}\mathcal{R}^2(x_0), \\ \frac{c_1}{2}(\eta + \sqrt{\eta^2 + \frac{4\theta}{c_1\zeta}}) &< \varepsilon < f(x_0) - f^*, \\ u &:= \frac{c_1}{2} \left(\eta + \sqrt{\eta^2 + \frac{4\theta}{c_1}} \right), \\ K &:= \frac{c_1}{\varepsilon - u} + \frac{c_1}{\varepsilon - \eta c_1} \log \left(\frac{\varepsilon - \frac{\theta c_1}{\varepsilon - \eta c_1}}{\varepsilon \zeta - \frac{\theta c_1}{\varepsilon - \eta c_1}} \right) + 2, \end{aligned}$$

then $\mathbb{P}(f(x_K) - f^* \leq \varepsilon) \geq 1 - \zeta$. Note that:

- Our η and θ are α and β in their notation.
- In (Tappenden et al., 2016) $c_1 = 2\mathcal{R}_{\ell_{p-1}}^2(x_0)$ where $\mathcal{R}_{\ell_{p-1}}^2(x_0)$ is defined as in (2) but using a norm $\|\cdot\|_{\ell_{p-1}}$ instead of $\|\cdot\|_2$. These norms are related as:

$$\begin{aligned} \|x\|_{\ell_{p-1}}^2 &= \sum_{j=1}^J \frac{L_j}{p_j} \|x^{(j)}\|_2^2 \stackrel{(a)}{=} \sum_{j=1}^J JL_{\max} \|x^{(j)}\|_2^2 \\ &= JL_{\max} \|x\|_2^2 \end{aligned}$$

where (a) follows as $p_j = 1/J$ for all j and we are taking $L_j = L_{\max}$. Hence, $c_1 = 2JL_{\max}\mathcal{R}^2(x_0)$ as stated.

- $K = \tilde{\mathcal{O}}(J/\varepsilon)$.

Replace η and θ with the expressions given by Lemma C.2 to obtain the expressions given in the statement of Theorem 3.1. Because (9) holds with high probability at each iteration, by the union bound it holds for all K iterations with probability greater than

$$\begin{aligned} &1 - K\mathcal{O} \left(J \exp \left(-\frac{0.01s_{\text{exact}}}{3J} \right) \right) \\ &= 1 - \tilde{\mathcal{O}} \left(\frac{J^2}{\varepsilon} \exp \left(-\frac{0.01s_{\text{exact}}}{3J} \right) \right) \end{aligned}$$

for ZO-BCD-R and with probability greater than

$$\begin{aligned} &1 - 2JK \exp \left(\frac{-0.01s_{\text{exact}}}{3J} \right) - (d/J)^{\log(d/J) \log^2(4.4s/J)} \\ &= 1 - \tilde{\mathcal{O}} \left(\frac{J^2}{\varepsilon} \exp \left(-\frac{0.01s_{\text{exact}}}{3J} \right) \right) - (d/J)^{\log(d/J) \log^2(4.4s/J)} \end{aligned} \quad (12)$$

Note the third term in (12) is universal, *i.e.* holds for all iterations, and hence is not multiplied by K . In both cases

we use $K = \tilde{\mathcal{O}}(J/\varepsilon)$. As stated, if (9) holds for all k then $\mathbb{P}(f(x_K) - f^* \leq \varepsilon) \geq 1 - \zeta$. Apply the union bound again to obtain the probabilities given in the statement of Theorem 3.1. ZO-BCD-R uses $m = 1.1b_1(s/J) \log(d/J)$ queries per iteration, all made by the GradientEstimation subroutine, and hence makes:

$$mK = (1.1b_1(s/J) \log(d/J)) K = \tilde{\mathcal{O}}(s/\varepsilon)$$

queries in total, using $K = \tilde{\mathcal{O}}(J/\varepsilon)$. On the other hand, ZO-BCD-RC makes $m = 1.1b_3(s/J) \log^2(1.1s/J) \log^2(d/J)$ queries per iteration. A similar calculation reveals that ZO-BCD-RC also makes $mK = \tilde{\mathcal{O}}(s/\varepsilon)$ queries in total.

The computational cost of each iteration is dominated by solving the sparse recovery problem using CoSaMP. It is known (Needell & Tropp, 2009) that CoSaMP requires $\mathcal{O}(n\mathcal{T})$ FLOPS, where $\mathcal{T}$ is the cost of a matrix-vector multiply by Z . For ZO-BCD-R $Z \in \mathbb{R}^{m \times (d/J)}$ is dense and unstructured hence:

$$\mathcal{T} = \mathcal{O} \left(m \frac{d}{J} \right) = \mathcal{O} \left(\frac{s}{J} \log(d/J) \frac{d}{J} \right) = \tilde{\mathcal{O}} \left(\frac{sd}{J^2} \right).$$

As noted in (Needell & Tropp, 2009), n may be taken to be $\mathcal{O}(1)$ (In all our experiments we take $n \leq 10$). For ZO-BCD-RC, as Z is a partial circulant matrix, we may exploit a fast matrix-vector multiplication via fast Fourier transform to reduce the complexity to $\mathcal{O}(d/J \log(d/J)) = \tilde{\mathcal{O}}(d/J)$.

Finally, we note that for both variants the memory complexity of ZO-BCD is dominated by the cost of storing Z . Again, as Z is dense and unstructured in ZO-BCD-R there are no tricks that one can exploit here, so the storage cost is $m(d/J) = \tilde{\mathcal{O}}(sd/J^2)$. For ZO-BCD-RC, instead of storing the entire partial circulant matrix Z , one just needs to store the generating vector $z \in \mathbb{R}^{d/J}$ and the index set Ω . Assuming we are allocating 32 bits per integer, this requires:

$$\frac{d}{J} + 32 \cdot b_3 \frac{s}{J} \log^2 \left(\frac{s}{J} \right) \log^2 \left(\frac{d}{J} \right) = \mathcal{O} \left(\frac{d}{J} \right).$$

This finishes the proof. $\square$

D. Experimental setup details

In this section, we provide the detailed experimental settings for the numerical results provided in Section 5.

D.1. Settings for synthetic experiments

For both synthetic examples, we use problem dimension $d = 20,000$ and gradient sparsity $s = 200$. Moreover, we use additive Gaussian noise with variance $\sigma = 10^{-3}$ in the oracles. The sampling radius is chosen to be 10^{-2} for all tested algorithms. For ZO-BCD, we use 5 blocks with

uniform step size⁸ $\alpha = 0.9$, and the per block sparsity is set to be $s_{\text{block}} = 1.05s/J = 42$. Furthermore, the block gradient estimation step runs at most $n = 10$ iterations of CoSaMP. For the other tested algorithms, we hand tune the parameters for their best performance, and same step sizes are used when applicable. Note that SPSA must use a very small step size ($\alpha = 0.01$) in max- s -squared-sum problem, or it will diverge.

Re-shuffling the blocks. Note that the max- s -squared-sum function does not satisfy the Lipschitz differentiability condition (*i.e.* Assumption 1). Moreover the gradient support changes, making this an extremely difficult zeroth-order problem. To overcome the difficulty of discontinuous gradients, we apply an additional step that re-shuffles the blocks every J iterations. This re-shuffling trick is not required for the problems that satisfies our assumptions; nevertheless, we observe very similar convergence behavior with slightly more queries when the re-shuffling step was applied on the problems that satisfy the aforementioned assumptions.

D.2. Settings for sparse DWT attacks on images

We allow a total query budget of 10,000 for all tested algorithms in each image attack, *i.e.* an attack is considered a failure if it cannot change the label within 10,000 queries. We use a 3-level ‘db45’ wavelet transform. All the results present in Section 5.2 use the half-point symmetric boundary extension, hence the wavelet domain has a dimension of 676,353; slightly larger than the original pixel domain dimension. For the interested reader, a discussion and more results about other boundary extensions can be found in Appendix E.

For all variations of ZO-BCD, we choose the block size to be 170 with per block sparsity $s_{\text{block}} = 10$, thus $m = 52$ queries are used per iteration. Sampling radius is set to be 10^{-2} . The block gradient estimation step runs at most $n = 30$ CoSaMP iterations, and step size $\alpha = 10$.

D.3. Settings for sparse CWT attacks on audio signals

For both targeted and untargeted CWT attack, we use Morse wavelets with 111 frequencies, which significantly enlarges the problem dimension from 16,000 to 1,776,000 per clip. In the attack, we choose the block size to be 295 with per block sparsity $s_{\text{block}} = 9$, thus $m = 52$ queries are used per iteration. The sampling radius is $\delta = 10^{-3}$. The block gradient estimation step runs at most $n = 30$ CoSaMP iterations. In targeted attacks, the step size is $\alpha = 0.05$, and Table 4 specifies the step size used in untargeted attacks.

⁸We note that using a line search for each block would maximize the advantage of block coordinate descent algorithms such as ZO-BCD, but we did not do so here for fairness

In Table 4, we also include a result of voice domain attack for the comparison. The parameter settings are same with aforementioned untargeted CWT attack settings, but we have to reduce step size to 0.01 for stability. Also, note that the problem domain is much smaller in the original voice domain, so the number of blocks is much less while we keep the same block size.

E. More experimental results and discussion

E.1. Sparse DWT attacks on images

Periodic extension. As mentioned, when we use boundary extensions other than periodic extension, the dimension of the wavelet coefficients will increase, depending on the size of wavelet filters and the level of the wavelet transform. More precisely, wavelets with larger support and/or deeper levels of DWT result in a larger increase in dimensionality. On the other hand, using periodic extension will not increase the dimension of wavelet domain. We provide test results for both boundary extensions in this section for the interested reader.

Compressed attack. As discussed in Section 4, DWT is widely used in data compression, such as JPEG-2000 for image. In reality, the compressed data are often saved in the form of sparse (and larger) DWT coefficients after vanishing the smaller ones. While we have already tested an attack on the larger DWT coefficients only (see Section 5.2), it is also of interest to test an attack after compression. That is, we zero out the smaller wavelet coefficients ($\text{abs} \leq 0.05$) first, and then attack on only the remaining, larger coefficients.

We use the aforementioned parameter settings for these two new attacks. We present the results in Table 6, and for completeness we also include the results already presented in Table 2. One can see that ZO-BCD-R(compressed) has higher attack success rate and lower ℓ_2 distortion, exceeding the prior state-of-the-art as presented in Table 2. We caution that this is not exactly a fair comparison with prior works however, as the compression step modifies the image before the attack begins.

Defense tests. Finally, we also tested some simple mechanisms for defending against our attacks; specifically harmonic denoising. We test DWT with the famous Haar wavelets, DWT with db45 wavelets which is also used for attack, and the essential discrete cosine transform (DCT). The defense mechanism is to apply the transform to the attacked images and then denoise by zeroing out small wavelet coefficients before transforming back to the pixel domain. We only test the defense on images that were successfully attacked. Tables 7 and 8 show the results of defending against the ZO-BCD-R and ZO-BCD-R(large coeff.) attacks respectively. Interestingly, using the attack wavelet (*i.e.* db45) in

Table 6. Results of untargeted image adversarial attack using various algorithms. Attack success rate (ASR), average final ℓ_2 distortion, and number of queries till first successful attack. ZO-BCD-R(periodic ext.) stands for ZO-BCD-R applying periodic extension for implementing the wavelet transform. ZO-BCD-R(compressed) stands for applying ZO-BCD-R to attack only large wavelet coefficients ($\text{abs} \geq 0.05$) and vanishing the smaller values. The other methods are the same in Table 2.

METHODS	ASR	ℓ_2 DIST	QUERIES
ZO-SCD	78%	57.5	2400
ZO-SGD	78%	37.9	1590
ZO-AdaMM	81%	28.2	1720
ZORO	90%	21.1	2950
ZO-BCD-R	92%	14.1	2131
ZO-BCD-RC	92%	14.2	2090
ZO-BCD-R(periodic ext.)	95%	21.0	1677
ZO-BCD-R(compressed)	96%	13.1	1546
ZO-BCD-R(large coeff.)	96%	13.7	1662

Table 7. Defense of image adversarial wavelet attack by ZO-BCD-R. Defense recovery success rate under haar and db45 wavelet filters, and discrete cosine transform (DCT) filter. Thresholding values of 0.05, 0.10, 0.15, and 0.20 were considered.

DEFENCE METHODS	0.05	0.10	0.15	0.20
Haar	74%	75%	76%	75%
db45	74%	72%	71%	63%
DCT	72%	79%	75%	67%

defence is not a good strategy. We obtain the best defense results by using a mismatched transform (*i.e.* DCT or Haar defense for a db45 attack) and a small thresholding value.

More adversarial examples. In Figure 5, we presented some adversarial images generated by the ZO-BCD-R and ZO-BCD-R(large coeff.) attacks. For the interested reader, we present more visual examples in Figure 7, and include the adversarial attack results generated by all versions of ZO-BCD.

E.2. Sparse CWT attacks on audio signals

Adversarial attacks on speech recognition is a more nebulous concept than that of adversarial attacks on image classifiers, with researchers considering a wide variety of threat models. In (Cisse et al., 2017), an attack on the speech-to-text engine DeepSpeech (Hannun et al., 2014) is successfully conducted, although the proposed algorithm, Houdini, is only able to force minor mis-transcriptions (“A great saint, saint Francis Xavier” becomes “a green thanked saint fredstus savia”). In (Carlini & Wagner, 2018), this problem is revisited, and they are able to achieve 100% success in targeted attacks, with any initial and target transcription

Table 8. Defense of image adversarial wavelet attack by ZO-BCD-R (large coeff.). Defense recovery success rate under Haar and db45 wavelet filters, and discrete cosine transform (DCT) filter. Thresholding values 0.05, 0.10, 0.15, and 0.20 were considered.

DEFENCE METHODS	0.05	0.10	0.15	0.20
Haar	75%	72%	78%	76%
db45	71%	70%	69%	63%
DCT	62%	74%	68%	58%

from the Mozilla Common Voices dataset (for example, “it was the best of times, it was the worst of times” becomes “it is a truth universally acknowledged that a single”). We emphasize that both of these attacks are *whitebox*, meaning that they require full knowledge of the victim model. We also note that speech-to-text transcription is not a classification problem, thus the classic Carlini-Wagner loss function so frequently used in generating adversarial examples for image classifiers cannot be straightforwardly applied. The difficulty of designing an appropriate attack loss function is discussed at length in (Carlini & Wagner, 2018).

A line of research more related to the current work is that of attacking keyword classifiers. Here, the victim model is a classifier; designed to take as input a short audio clip and to output probabilities of this clip containing one of a small, predetermined list of keywords (“stop”, “left” and so on). Most such works consider the SpeechCommands dataset (Warden, 2018). To the best of the author’s knowledge, the first paper to consider targeted attacks on a keyword classification model was (Alzantot et al., 2018), and they do so in a black-box setting. They achieve an 87% attack success rate (ASR) using a genetic algorithm, whose query complexity is unclear. They do not report the relative loudness of their attacks; instead they report the results of a human study in which they asked volunteers to listen to and label the attacked audio clips. They report that for 89% of the successfully attacked clips human volunteers still assigned the clips the correct (*i.e.* source) label, indicating that these clips were not corrupted beyond comprehension. Their attacks are per-clip, *i.e.* not universal.

In (Vadillo & Santana, 2019), universal and untargeted attacks on a SpeechCommands classifier are considered. Specifically, they seek to construct a distortion δ such that for *any* clip x from a specified source class (*e.g.* “left”), the attacked clip $x + \delta$ is misclassified to a source class (*e.g.* “yes”) by the model. They consider several variations on this theme; allowing for multiple source classes. The results we recorded in Section 5.3 (ASR of 70.3% at a remarkably low mean relative loudness of -41.63 dB) were the best reported in the paper, and were for the single-source-class setting. This attack was in the white-box setting.

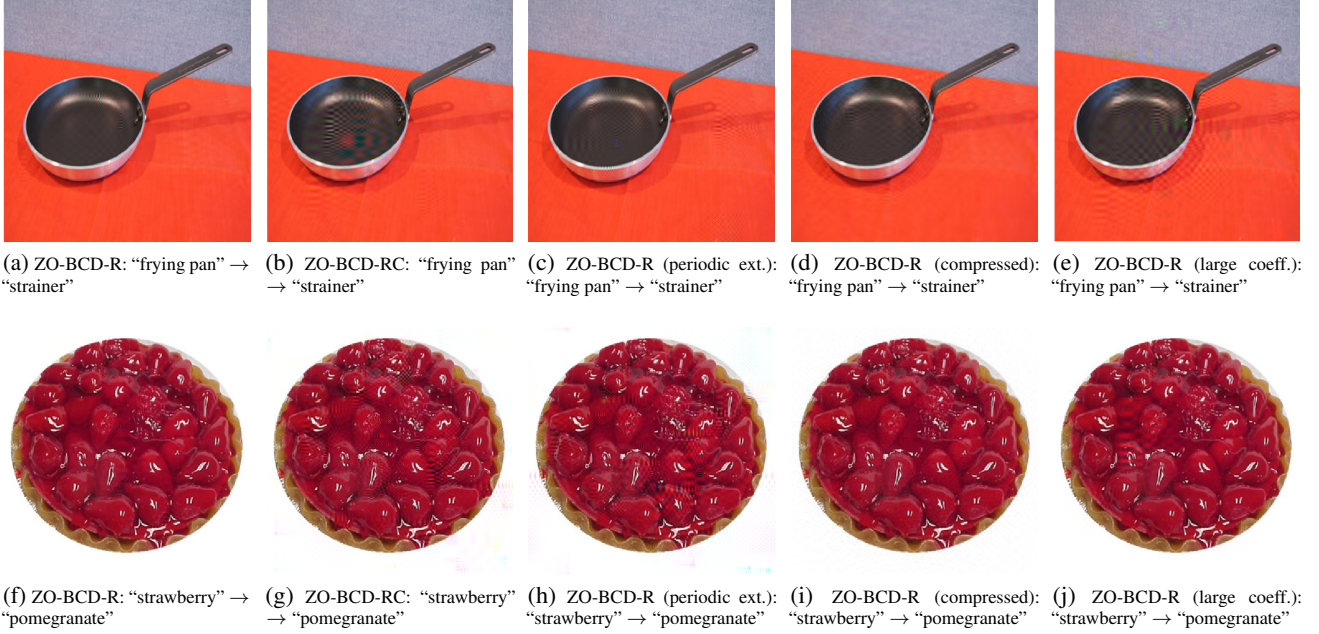


Figure 7. More examples of wavelet attacked images by ZO-BCD-R, ZO-BCD-RC, ZO-BCD-R(periodic ext.), ZO-BCD-R(compressed), and ZO-BCD-R(large coeff.), with true labels and mis-classified labels.

Finally, we mention two recent works which consider very interesting, but different threat models. (Li et al., 2020) considers the situation where a malicious attacker wishes to craft a short (say 0.5 second long) that can be added to *any part* of a clean audio clip to force a misclassification. Their attacks are targeted and universal, and conducted in the white-box setting. They do not report the relative loudness of their attacks. In (Xie et al., 2020), a generative model is trained that takes as input a benign audio clip x , and returns an attacked clip $x + \delta$. The primary advantage of this approach is that attacks can be constructed on the fly. In the targeted, per-clip white-box setting they achieve the success rate of 93.6% advertised in Section 5.3, at an approximate relative loudness of -30 dB. They also consider universal attacks, and a transfer attack whereby the generative model is trained on a surrogate classification model.

In all the aforementioned keyword attacks, the victim model is some variant of the model proposed in (Sainath & Parada, 2015). Specifically, the audio input is first transformed into a 2D spectrogram using Mel frequency coefficients, bark coefficients or similar. Then, a 3- to 5- layer convolutional neural network is applied.